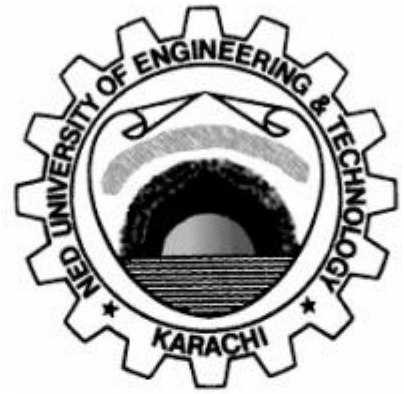
Complex Engineering Activity Report

Object Oriented Programming

First Year-Software Engineering

Batch: 2025



Group Members:

TALHA SIDDIQUI …… SE – 25096
MUHAMMAD KAYHAN KHAN …. SE -25095
IMRAN QADIR …. SE -25080

Submitted to: Dr. Saad Qasim Khan

Submission Date: 04/05/2026

NED University of Engineering & Technology

Department of Software Engineering

Object-Oriented Programming — C++ Project Report

First Year | Second Semester | Spring 2026

ONLINE SHOPPING SYSTEM

Using Object-Oriented Programming in C++

Submitted By:

Talha Siddiqui (SE-25096)
Muhammad Kayhan Khan (SE-25095)
Imran Qadie (SE-25080)

Submitted To:

Sir Saad Qasim

MAY 2026

TABLE OF CONTENTS

1. ABSTRACT

In today's world, people prefer to shop online instead of visiting physical stores. However, managing products, users, carts, and orders in a structured way requires a well-designed system. This project presents a console-based Online Shopping System built in C++ that allows customers to browse products, add them to a cart, and place orders with payment options. The system is developed using core Object-Oriented Programming concepts to make the code organized, reusable, and easy to understand.

The project implements fourteen Object-Oriented Programming concepts: Abstract Class, Inheritance, Runtime Polymorphism, Compile-Time Polymorphism (Function Overloading), Encapsulation, Composition, Aggregation, Friend Function, Struct, Constructors, Destructors, STL vector, and Dynamic Memory Management. All concepts are applied in a single, well-commented C++ source file.

Constructor and destructor messages are printed to the console at run-time, making the lifecycle of every object clearly visible. The menu-driven interface provides seven operations for the user to interact with the system. The code is written to be simple and beginner-friendly, making it ideal for a first-year Software Engineering course.

2. OOP CONCEPTS — DETAILED OVERVIEW

This section provides a clear explanation of every Object-Oriented Programming concept used in this project. Each concept is explained in simple English to serve as a reference for first-year students.

1. Abstract Class: An abstract class is a class that cannot be instantiated on its own. It exists only to serve as a base for other classes. A class becomes abstract when it contains at least one pure virtual function — declared with = 0. In this project, the class Product is abstract because it declares: virtual void displayProduct() = 0;. This forces every derived class (Electronics, Clothing) to provide its own implementation before any object of that type can be created.

2. Inheritance: Inheritance is the OOP mechanism by which one class (child/derived) acquires the properties and behaviours of another class (parent/base). This project uses two separate inheritance chains. First, the class Product is the abstract base, from which Electronics and Clothing are derived — these inherit productID, name, and price. Second, the class User is the base, from which Customer and Admin are derived — these inherit username and email.

3. Runtime Polymorphism: Runtime polymorphism (also called dynamic dispatch) means the correct version of an overridden function is selected at run-time, based on the actual type of the object, not the type of the pointer. In this project, a vector<Product*> holds pointers to Electronics and Clothing objects. When displayProduct() is called through any of these pointers, C++ automatically calls the correct version — Electronics::displayProduct() or Clothing::displayProduct() — even though the pointer is of type Product*.

4. Compile-Time Polymorphism: Compile-time polymorphism (also called static dispatch) is achieved through Function Overloading. Function overloading means defining two or more functions with the same name but different parameter types. The compiler selects the correct version at compile time based on the argument provided. In this project, the Cart class has two versions of addToCart(): one accepts an int (product ID) and the other accepts a string (product name). The compiler chooses the correct version at the point of each call.

5. Encapsulation: Encapsulation means bundling data (attributes) and the functions that operate on them together inside a class, and restricting direct access to internal data by making it private or protected. Public getter and setter functions provide controlled access. In this project, every class uses encapsulation: for example, totalAmount in Payment is private, and productID, name, price in Product are protected. No outside code can modify these values directly.

6. Composition: Composition is a strong HAS-A relationship where one object contains another as a direct member. The contained object's lifetime is tied to the container — it is created when the container is created and destroyed when the container is destroyed. In this project, the class Order

contains a Payment object as a direct data member (not a pointer). This means the Payment object cannot exist independently of the Order.

7. Aggregation: Aggregation is a weak HAS-A relationship where one object stores references or pointers to other objects that exist independently. The contained objects are not owned by the container — they can outlive it. In this project, the Cart class stores a vector<Product*> — pointers to Product objects that are created in main() and exist independently. If the Cart is destroyed, the Products continue to exist.

8. Friend Function: A friend function is a function that is declared inside a class with the keyword friend, which grants it direct access to the private and protected members of that class, even though the function is not a member of the class. In this project, the global function applyDiscount(Payment& pay, double percent) is a friend of the Payment class. It can directly read and modify the private member totalAmount without using a public setter.

9. Structure (struct): A struct in C++ is a user-defined data type that groups related variables together under a single name. Unlike a class, all members of a struct are public by default. Structs are used for simple data containers that do not need methods or access control. In this project, struct OrderDetails holds two fields: orderID (int) and date (string). This struct is used as a data member inside the Order class.

10. Constructors: A constructor is a special member function that is automatically called when an object is created. It has the same name as the class and no return type. Constructors are used to initialise the object's data members to valid starting values. In this project, every class has a parameterized constructor. Each constructor also prints a message to the console (e.g., [Constructor] Product created: ...) so students can see exactly when each object is created.

11. Destructors: A destructor is a special member function that is automatically called when an object goes out of scope or is explicitly deleted. It has the same name as the class prefixed with a tilde (~) and no parameters or return type. In this project, every class has a destructor that prints a message. The classes Product and User use virtual destructors, which is essential when deleting derived-class objects through base-class pointers.

12. STL Vector: The Standard Template Library (STL) vector is a dynamic array that can grow or shrink at runtime. It is defined in the <vector> header. In this project, vector<Product*> is used in Cart to hold cart items and in main() to hold the product catalogue. vector<string> is used in Order to store the list of ordered item names. Unlike plain arrays, vectors manage their own memory automatically.

13. Dynamic Memory (new / delete): Dynamic memory management means allocating memory on the heap at run-time using new and freeing it using delete. This is necessary when the number of

objects is not known at compile time, or when objects need to outlive the scope in which they were created. In this project, all six Product objects are created with new in main() and stored in vector<Product*>. They are explicitly freed in the cleanup block using a delete loop.

3. INTRODUCTION

The Online Shopping System is a menu-driven, console-based C++ application that simulates the complete workflow of an e-commerce platform. The project was developed as a major Object-Oriented Programming assignment for first-year Software Engineering students at NED University of Engineering and Technology, Karachi. The goal of this project is to demonstrate how real-world systems are designed and structured using the principles of OOP in C++.

3.1 Problem Statement

Traditional software written in procedural C lacks the structure and flexibility required for modern, scalable applications. The following problems arise when shopping system logic is written procedurally:

- There is no structure to group related data and functions together, making the code difficult to maintain and extend.
- Adding a new product type (for example, Books) requires changes throughout the entire codebase rather than simply adding one new class.
- There is no way to model real-world relationships such as HAS-A (a cart has products, an order has a payment) or IS-A (a customer is a user).
- Memory management is error-prone without the automatic constructor/destructor lifecycle provided by classes.
- Sensitive data such as payment amounts cannot be properly protected in a purely procedural design.
- Code reuse is minimal — common attributes such as username and email must be duplicated across multiple modules.

3.2 Proposed Solution

The Online Shopping System solves these problems by applying a carefully designed OOP class hierarchy. Each real-world entity (Product, User, Cart, Order, Payment) is modelled as a separate class with its own data and responsibilities. The specific design decisions are:

- Product is designed as an abstract class so that no generic product can be created — only specific types (Electronics, Clothing) are instantiable. This enforces correct usage.
- Inheritance is used so that Electronics and Clothing automatically get all common product attributes without duplicating code.
- A vector<Product*> allows mixed collections of Electronics and Clothing objects to be stored together and iterated polymorphically.
- Composition is used for Order and Payment because a payment logically belongs to an order and should not exist independently.

- Aggregation is used for Cart and Product because products exist in the store independently of any particular cart.
- Encapsulation protects sensitive data (totalAmount, productID) behind public interfaces, preventing accidental corruption.

3.3 Project Objectives

- Implement all fourteen required OOP concepts in a single, cohesive, well-commented C++ console application.
- Model the complete online shopping workflow: browse products, add to cart by ID or by name, place an order, and apply a discount.
- Demonstrate runtime polymorphism visibly using a vector of abstract base-class pointers.
- Show the structural difference between Composition (Order owns Payment) and Aggregation (Cart references Products).
- Provide clear inline comments next to every OOP concept as it is applied in the source code.
- Make the constructor and destructor lifecycle fully visible on the console to help students understand object creation and destruction.
- Keep all code simple and beginner-friendly, suitable for a first-year Software Engineering audience.

3.4 Scope of the Project

In Scope:

- Console-based menu interface with seven operations.
- Product catalogue with Electronics (brand attribute) and Clothing (size attribute) categories.
- Customer and Admin user types derived from a common User base class.
- Cart supporting add-by-ID and add-by-name operations with total calculation.
- Order placement with embedded Payment object, selectable payment method, and optional discount.
- Full dynamic memory allocation (new) and cleanup (delete) for all product objects.
- Visible constructor and destructor messages for all nine classes.

Out of Scope:

- Graphical user interface (GUI) — this is a console application only.
- File-based or database persistence — all data is in-memory and lost on exit.
- User login authentication with passwords.
- Network connectivity or online payment gateway integration.

4. SYSTEM ARCHITECTURE

The system is built around nine classes organised into two inheritance hierarchies, one composition relationship, and one aggregation relationship. All classes are implemented in a single .cpp file with clearly labelled sections. The architecture is intentionally straightforward so that every OOP relationship is easy to identify and study.

4.1 Overall Class Design

The nine classes and their relationships are summarised in the table below.

Class	Type	Key Attributes	Key Methods
Product	*Abstract Base*	`productID, name, price`	`displayProduct()=0, getID(), getName(), getPrice()`
Electronics	*Derived (Product)*	`brand`	`displayProduct() override`
Clothing	*Derived (Product)*	`size`	`displayProduct() override`
User	*Base Class*	`username, email`	`showProfile(), getUsername()`
Customer	*Derived (User)*	`address`	`showProfile() override`
Admin	*Derived (User)*	`adminCode`	`showProfile() override`
Payment	*Standalone*	`totalAmount, method`	`displayPayment(), getAmount(), getMethod()`
Order	*Composition*	`OrderDetails (struct), Payment, items`	`displayOrder(), getPayment()`
Cart	*Aggregation*	`vector<Product*> items`	`addToCart(int), addToCart(string), viewCart()`

4.2 Product Class Hierarchy

Product is the abstract base class at the top of the first hierarchy. It defines the common interface for all products through the pure virtual function displayProduct(). Electronics and Clothing are concrete derived classes. Each adds one extra attribute (brand and size respectively) and overrides displayProduct() with its own specific output format. The virtual destructor in Product ensures the correct destructor chain fires when products are deleted via Product* pointers.

4.3 User Class Hierarchy

User is the base class for the second hierarchy. It holds the common attributes username and email, and provides a virtual showProfile() function. Customer derives from User and adds an address

attribute. Admin derives from User and adds an adminCode attribute. Both derived classes override showProfile() to display their specific information while also calling the base class version using User::showProfile().

4.4 Cart, Order, and Payment Design

Cart demonstrates Aggregation: it stores a vector<Product*> of pointers to Product objects owned by main(). Adding a product to the cart simply pushes its pointer into the vector — no ownership is transferred. The Cart provides two overloaded addToCart() versions (by ID and by name), demonstrating compile-time polymorphism.

Order demonstrates Composition: it contains a Payment object and an OrderDetails struct as direct data members. The Payment and struct are initialised in the Order constructor's initialiser list and are destroyed automatically when the Order is destroyed. The Order class also stores a vector<string> of ordered item names for display in the order summary.

5. IMPLEMENTATION — EVERY OOP CONCEPT WITH CODE

This section presents the complete source code for each OOP concept alongside a detailed explanation. The code shown is taken directly from the submitted OnlineShoppingSystem.cpp file.

5.1 Abstract Class and Pure Virtual Function

An abstract class is created by declaring at least one pure virtual function inside the class. A pure virtual function is written as: virtual returnType functionName() = 0;. The = 0 part tells the compiler that this class does not provide an implementation — it is the responsibility of every derived class to provide one. Any class that contains a pure virtual function cannot be instantiated directly. Attempting to write Product p(...) would cause a compile-time error.

Source Code — Abstract Class Product:

```
// ============================================================
// SECTION 2: ABSTRACT BASE CLASS — Product
// OOP Concept: Abstract Class + Runtime Polymorphism
// Pure virtual function forces derived classes to override it
// ============================================================
class Product {
protected:
    int    productID;   // Encapsulation: protected data
    string name;
    double price;

public:
    // Parameterized Constructor
    Product(int id, string n, double p)
        : productID(id), name(n), price(p) {
        cout << "[Constructor] Product created: " << name << "\n";
    }

    // Pure Virtual Function — makes Product an Abstract Class
    // OOP Concept: Polymorphism (Runtime)
    virtual void displayProduct() = 0;

    // Getters — OOP Concept: Encapsulation
    int    getID()    const { return productID; }
    string getName()  const { return name; }
    double getPrice() const { return price; }

    // Virtual Destructor — essential when using base class pointers
    virtual ~Product() {
        cout << "[Destructor] Product destroyed: " << name << "\n";
    }
};
```

Key points to note: (1) The class Product cannot be instantiated — Product p(101, "Laptop", 50000) would fail at compile time. (2) The = 0 syntax is what makes displayProduct() a pure virtual function. (3) The virtual keyword on the destructor ensures that when delete is called on a Product* pointer

pointing to an Electronics or Clothing object, both the derived class destructor and the Product destructor are called in the correct order.

5.2 Inheritance — Product Hierarchy

Inheritance is declared using a colon (:) and an access specifier (public, protected, private) after the class name. Using public inheritance means all public members of the base class remain public in the derived class. Both Electronics and Clothing inherit the three protected attributes of Product (productID, name, price) and the three public getters (getID, getName, getPrice). They each add one new private attribute.

Source Code — Electronics (Derived from Product):

```
// ============================================================
// SECTION 3: DERIVED CLASS — Electronics
// OOP Concept: Inheritance (from Product)
//              Polymorphism (displayProduct overridden)
// ============================================================
class Electronics : public Product {   // Inheritance: Electronics IS-A Product
private:
    string brand;      // Extra attribute for Electronics

public:
    // Constructor — calls base class constructor using initialiser list
    Electronics(int id, string n, double p, string b)
        : Product(id, n, p), brand(b) {
        cout << "[Constructor] Electronics created: " << name << "\n";
    }

    // Overrides the pure virtual function from Product
    void displayProduct() override {
        cout << "  [Electronics] ID: " << productID
             << " | Name: "  << name
             << " | Brand: " << brand
             << " | Price: Rs." << price << "\n";
    }

    ~Electronics() {
        cout << "[Destructor] Electronics destroyed: " << name << "\n";
    }
};
```

Source Code — Clothing (Derived from Product):

```
// ============================================================
// SECTION 4: DERIVED CLASS — Clothing
// OOP Concept: Inheritance (from Product)
//              Polymorphism (displayProduct overridden)
// ============================================================
class Clothing : public Product {   // Inheritance: Clothing IS-A Product
private:
    string size;       // Extra attribute for Clothing

public:
```

```
    Clothing(int id, string n, double p, string s)
        : Product(id, n, p), size(s) {
        cout << "[Constructor] Clothing created: " << name << "\n";
    }

    void displayProduct() override {
        cout << "  [Clothing]     ID: " << productID
             << " | Name: " << name
             << " | Size: " << size
             << " | Price: Rs." << price << "\n";
    }

    ~Clothing() {
        cout << "[Destructor] Clothing destroyed: " << name << "\n";
    }
};
```

Notice that : Product(id, n, p) in the constructor initialiser list explicitly calls the base class constructor, passing productID, name, and price up to Product for initialisation. This is how derived classes initialise inherited attributes.

5.3 Runtime Polymorphism

Runtime polymorphism is demonstrated in two places: (1) in the allProducts vector in main() which stores Product* pointers pointing to actual Electronics and Clothing objects, and (2) in the Cart's viewCart() method which iterates through Product* pointers and calls displayProduct() on each.

Source Code — Runtime Polymorphism in main() and viewCart():

```
// In main() — creating the product catalogue using base-class pointers
vector<Product*> allProducts;
allProducts.push_back(new Electronics(101, "Samsung Galaxy A55", 89999,
"Samsung"));
allProducts.push_back(new Electronics(102, "HP Laptop 15s",     120000, "HP"));
allProducts.push_back(new Electronics(103, "JBL Bluetooth Speaker", 8500,
"JBL"));
allProducts.push_back(new Clothing(201, "Linen Kurta",   2500, "M"));
allProducts.push_back(new Clothing(202, "Denim Jeans",   4200, "L"));
allProducts.push_back(new Clothing(203, "Winter Hoodie", 3800, "XL"));


// Menu Option 1 — View Products
// OOP Concept: Runtime Polymorphism
// The pointer type is Product* but the actual objects are Electronics and
Clothing.
// C++ calls the correct displayProduct() version at run-time.
for (Product* p : allProducts) {
    p->displayProduct();  // Correct version called at runtime!
}


// In Cart::viewCart() — same polymorphism applies to cart items
for (Product* p : items) {
    p->displayProduct();  // Calls Electronics or Clothing version automatically
    total += p->getPrice();
}
```

The key insight is that the pointer p is of type Product*, but C++ uses the virtual function table (vtable) to look up the correct function implementation at run-time. Without the virtual keyword, this would not work — the base class version (which does not exist, since it is pure virtual) would be called instead.

5.4 Inheritance — User Hierarchy

The second inheritance hierarchy follows the same pattern as the Product hierarchy. The User base class provides common attributes and a virtual showProfile() function. Customer and Admin each override showProfile() and call User::showProfile() to include the base class output before their own specific information.

Source Code — User Base Class and Derived Customer and Admin:

```
// ============================================================
// SECTION 9: BASE CLASS — User
// OOP Concept: Inheritance (base class) + Encapsulation
// ============================================================
class User {
protected:
    string username;   // Encapsulation: accessible to derived classes
    string email;

public:
    User(string u, string e) : username(u), email(e) {
        cout << "[Constructor] User created: " << username << "\n";
    }
    virtual void showProfile() const {
        cout << "  Username : " << username << "\n";
        cout << "  Email    : " << email    << "\n";
    }
    string getUsername() const { return username; }
    virtual ~User() {
        cout << "[Destructor] User destroyed: " << username << "\n";
    }
};

// ============================================================
// SECTION 10: DERIVED CLASS — Customer
// OOP Concept: Inheritance (from User)
// ============================================================
class Customer : public User {
private:
    string address;
public:
    Customer(string u, string e, string addr)
        : User(u, e), address(addr) {
        cout << "[Constructor] Customer profile created: " << username << "\n";
    }
    void showProfile() const override {
        cout << "\n--- Customer Profile ---\n";
        User::showProfile();   // Call base class version first
        cout << "  Address  : " << address << "\n";
    }
```

```
    ~Customer() {
        cout << "[Destructor] Customer destroyed: " << username << "\n";
    }
};

// ============================================================
// SECTION 11: DERIVED CLASS — Admin
// OOP Concept: Inheritance (from User)
// ============================================================
class Admin : public User {
private:
    string adminCode;
public:
    Admin(string u, string e, string code)
        : User(u, e), adminCode(code) {
        cout << "[Constructor] Admin created: " << username << "\n";
    }
    void showProfile() const override {
        cout << "\n--- Admin Profile ---\n";
        User::showProfile();
        cout << "  Admin Code: " << adminCode << "\n";
    }
    ~Admin() {
        cout << "[Destructor] Admin destroyed: " << username << "\n";
    }
};
```

5.5 Compile-Time Polymorphism (Function Overloading)

Function overloading allows multiple functions with the same name to coexist in a class, as long as their parameter lists differ. The compiler identifies which version to call based on the type of argument passed at the point of each function call — this selection happens entirely at compile time, not at run-time.

Source Code — Two overloaded addToCart() versions in Cart:

```
class Cart {
private:
    vector<Product*> items;

public:
    // ============================================================
    // OOP Concept: Function Overloading (Compile-Time Polymorphism)
    // Version 1: Add product to cart by ID (int parameter)
    // ============================================================
    void addToCart(int id, vector<Product*>& allProducts) {
        for (Product* p : allProducts) {
            if (p->getID() == id) {
                items.push_back(p);
                cout << "   Added to cart by ID: " << p->getName() << "\n";
                return;
            }
        }
        cout << "   Product with ID " << id << " not found.\n";
    }
```

```
    // ============================================================
    // OOP Concept: Function Overloading (Compile-Time Polymorphism)
    // Version 2: Add product to cart by Name (string parameter)
    // ============================================================
    void addToCart(string name, vector<Product*>& allProducts) {
        for (Product* p : allProducts) {
            if (p->getName() == name) {
                items.push_back(p);
                cout << "   Added to cart by Name: " << p->getName() << "\n";
                return;
            }
        }
        cout << "   Product \"" << name << "\" not found.\n";
    }
};

// In main() — the compiler selects the correct version automatically:
cart.addToCart(101, allProducts);         // Calls version 1 (int)
cart.addToCart("Linen Kurta", allProducts); // Calls version 2 (string)
```

5.6 Encapsulation

Encapsulation is implemented consistently across all nine classes. Data members are declared private or protected, and public getter functions provide read-only access. The Payment class provides the clearest example of encapsulation — its totalAmount is private and can only be accessed through the public getter, except by the designated friend function.

Source Code — Encapsulation in Product and Payment:

```
// ============================================================
// OOP Concept: Encapsulation — Product class
// private/protected data + public getters only
// ============================================================
class Product {
protected:
    int    productID;   // Cannot be accessed from outside the class
    string name;        // or from non-derived classes
    double price;

public:
    // Read-only access via getters
    int    getID()    const { return productID; }
    string getName()  const { return name; }
    double getPrice() const { return price; }
    // No setters — these values should not change after construction
};

// ============================================================
// OOP Concept: Encapsulation — Payment class
// totalAmount is private and fully protected
// ============================================================
class Payment {
private:
    double totalAmount;   // Fully private — only accessible via getter
```

```
    string method;        // or via friend function applyDiscount()

public:
    double getAmount() const { return totalAmount; }
    string getMethod() const { return method; }

    void displayPayment() const {
        cout << "  Payment Method : " << method << "\n";
        cout << "  Total Amount   : Rs." << totalAmount << "\n";
    }
};
```

5.7 Composition

Composition is the strongest form of the HAS-A relationship. The contained object is a direct value member (not a pointer) of the container class. It is initialised in the container's constructor initialiser list and is automatically destroyed when the container is destroyed. There is no way to have the contained object without the container.

Source Code — Order class demonstrating Composition with Payment:

```
// ============================================================
// SECTION 7: CLASS — Order
// OOP Concept: Composition
//              Order HAS-A Payment object (lives inside Order)
//              Struct used for OrderDetails
// ============================================================
class Order {
private:
    OrderDetails   details;     // Struct as data member
    Payment        payment;     // COMPOSITION: Payment lives inside Order
    vector<string> itemNames;   // List of ordered item names

public:
    Order(int id, string date, double amount,
          string payMethod, vector<string> items)
        : payment(amount, payMethod), itemNames(items) {
        // Payment is initialised here in the initialiser list.
        // It cannot be created separately — it is part of Order.
        details.orderID = id;
        details.date    = date;
        cout << "[Constructor] Order #" << details.orderID << " created.\n";
    }

    void displayOrder() const {
        cout << "\n========== ORDER SUMMARY ==========\n";
        cout << "  Order ID  : " << details.orderID << "\n";
        cout << "  Date      : " << details.date    << "\n";
        cout << "  Items     :\n";
        for (const string& item : itemNames) {
            cout << "    -> " << item << "\n";
        }
        payment.displayPayment();
        cout << "====================================\n";
    }
```

```
    Payment& getPayment() { return payment; }

    ~Order() {
        cout << "[Destructor] Order #" << details.orderID << " destroyed.\n";
        // Payment's destructor fires automatically after this
    }
};
```

5.8 Aggregation

Aggregation is a weaker HAS-A relationship where the container stores pointers or references to objects that exist independently. In this project, the Cart stores Product* pointers. The Product objects are created in main() and stored in allProducts. The Cart only holds pointers to them — it does not own them. If the Cart is destroyed, the Products are unaffected.

Source Code — Cart class demonstrating Aggregation:

```
// ============================================================
// SECTION 8: CLASS — Cart
// OOP Concept: Aggregation
//              Cart stores Product* pointers (does NOT own them)
//              STL vector<Product*> used
// ============================================================
class Cart {
private:
    // AGGREGATION: Cart holds pointers to Product objects.
    // Products are created in main() and exist independently of Cart.
    // Cart only stores references (pointers) — not the objects themselves.
    vector<Product*> items;

public:
    Cart() { cout << "[Constructor] Cart created.\n"; }

    void viewCart() const {
        if (items.empty()) { cout << "  Your cart is empty!\n"; return; }
        cout << "\n---------- YOUR CART ----------\n";
        double total = 0;
        for (Product* p : items) {
            p->displayProduct();   // Runtime polymorphism here too!
            total += p->getPrice();
        }
        cout << "  Cart Total: Rs." << total << "\n";
    }

    double getTotal() const {
        double total = 0;
        for (Product* p : items) total += p->getPrice();
        return total;
    }

    vector<string> getItemNames() const {
        vector<string> names;
        for (Product* p : items) names.push_back(p->getName());
```

```
        return names;
    }

    bool isEmpty() const { return items.empty(); }

    ~Cart() {
        cout << "[Destructor] Cart destroyed.\n";
        // Products are NOT deleted here — Cart does not own them.
        // They are deleted separately in main().
    }
};
```

5.9 Friend Function

A friend function is granted special access to the private members of a class. It is declared inside the class using the friend keyword, but it is defined outside the class as a regular global function. In this project, applyDiscount() modifies the private totalAmount of Payment — something that no other non-member function can do.

Source Code — Friend function declaration in Payment and its definition:

```
// ============================================================
// SECTION 5: CLASS — Payment
// OOP Concept: Composition (used inside Order)
//              Friend Function (applyDiscount accesses private)
// ============================================================
class Payment {
private:
    double totalAmount;
    string method;

public:
    // FRIEND FUNCTION DECLARATION
    // This grants applyDiscount() access to private members
    friend void applyDiscount(Payment& pay, double discountPercent);

    Payment(double amount, string m) : totalAmount(amount), method(m) {
        cout << "[Constructor] Payment object created.\n";
    }
    double getAmount() const { return totalAmount; }
    string getMethod() const { return method; }
    void displayPayment() const {
        cout << "  Payment Method : " << method << "\n";
        cout << "  Total Amount   : Rs." << totalAmount << "\n";
    }
    ~Payment() { cout << "[Destructor] Payment object destroyed.\n"; }
};

// ============================================================
// SECTION 6: FRIEND FUNCTION — applyDiscount
// OOP Concept: Friend Function
//              Accesses private member 'totalAmount' of Payment
// ============================================================
void applyDiscount(Payment& pay, double discountPercent) {
    // Direct access to private member — only possible because this
    // function is declared a friend inside the Payment class.
    double discountAmount = (pay.totalAmount * discountPercent) / 100;
    pay.totalAmount -= discountAmount;   // Modifying private member!
    cout << "  Discount of " << discountPercent
         << "% applied! New Total: Rs." << pay.totalAmount << "\n";
}

// In main() — calling the friend function:
applyDiscount(order->getPayment(), disc);
```

5.10 Structure (struct)

A struct is used for simple, plain data storage where no methods or access control are needed. In C++, struct members are public by default. Structs are ideal for grouping a few related fields that are always used together. Here, OrderDetails groups the order ID and date so they can be passed and stored as a single unit.

Source Code — struct OrderDetails and its use inside Order:

```
// ============================================================
// SECTION 1: STRUCT
// OOP Concept: Structure to store basic order information
// ============================================================
struct OrderDetails {
    int    orderID;  // Unique ID for each order
    string date;     // Date of order placement
    // Members are public by default in a struct
};

// Usage inside Order class:
class Order {
private:
    OrderDetails details;   // OOP Concept: Struct usage
    // ...
public:
    Order(int id, string date, ...) : ... {
        details.orderID = id;    // Accessing struct members directly
        details.date    = date;
    }
    void displayOrder() const {
        cout << "  Order ID : " << details.orderID << "\n";
        cout << "  Date     : " << details.date    << "\n";
    }
};
```

5.11 Constructors

Every class in this project has a parameterized constructor that uses an initialiser list (the : syntax after the constructor parameters) to initialise its data members before the constructor body executes. Using an initialiser list is more efficient than assignment inside the constructor body. Each constructor also prints a confirmation message so the creation sequence is visible.

Source Code — Constructor examples from multiple classes:

```
// Product Constructor (base class constructor)
Product(int id, string n, double p)
    : productID(id), name(n), price(p) {    // Initialiser list
    cout << "[Constructor] Product created: " << name << "\n";
}

// Electronics Constructor (calls base class constructor)
Electronics(int id, string n, double p, string b)
    : Product(id, n, p), brand(b) {    // Passes id, n, p up to Product
    cout << "[Constructor] Electronics created: " << name << "\n";
}
```

```
// Payment Constructor
Payment(double amount, string m)
    : totalAmount(amount), method(m) {
    cout << "[Constructor] Payment object created.\n";
}

// Order Constructor — initialises Payment and itemNames in initialiser list
Order(int id, string date, double amount, string payMethod, vector<string> items)
    : payment(amount, payMethod), itemNames(items) {
    details.orderID = id;
    details.date    = date;
    cout << "[Constructor] Order #" << details.orderID << " created.\n";
}

// Customer Constructor — calls User base constructor
Customer(string u, string e, string addr)
    : User(u, e), address(addr) {
    cout << "[Constructor] Customer profile created: " << username << "\n";
}
```

5.12 Destructors

Every class has a destructor. For base classes (Product, User), the destructor is declared virtual. Without the virtual keyword, deleting a derived object through a base pointer would only call the base destructor, skipping the derived destructor and causing a memory leak or undefined behaviour. The virtual keyword ensures the complete destructor chain fires in the correct reverse order.

Source Code — Destructor chain for Electronics and Product:

```
// Virtual destructor in base class — CRITICAL for correct chain
virtual ~Product() {
    cout << "[Destructor] Product destroyed: " << name << "\n";
}

// Destructor in derived class
~Electronics() {
    cout << "[Destructor] Electronics destroyed: " << name << "\n";
    // When this finishes, C++ automatically calls Product::~Product()
}

// In main() — cleanup block:
for (Product* p : allProducts) {
    delete p;
    // For an Electronics object, this call fires in order:
    // 1. Electronics::~Electronics()  (derived destructor first)
    // 2. Product::~Product()           (base destructor second)
}

// Similarly for Composition — when Order is destroyed:
~Order() {
    cout << "[Destructor] Order #" << details.orderID << " destroyed.\n";
    // After this, Payment::~Payment() fires automatically
    // because payment is a direct member of Order
```

```
}

~Payment() {
    cout << "[Destructor] Payment object destroyed.\n";
}
```

5.13 STL Vector

The Standard Template Library vector is a resizable array that manages its own memory. It provides push_back() to add elements, size() to get the count, empty() to check if it is empty, and range-based for loops for iteration. In this project, three different vectors are used.

Source Code — Three uses of vector in this project:

```
#include <vector>
using namespace std;

// ── Use 1: Product catalogue in main() ──────────────────────
// Stores Product* pointers to Electronics and Clothing objects on heap
vector<Product*> allProducts;
allProducts.push_back(new Electronics(101, "Samsung Galaxy A55", 89999,
"Samsung"));
allProducts.push_back(new Clothing(201, "Linen Kurta", 2500, "M"));
// ...

// ── Use 2: Cart items in Cart class ─────────────────────────
// AGGREGATION: stores pointers to products owned by main()
class Cart {
private:
    vector<Product*> items;   // OOP Concept: Aggregation + STL vector
public:
    void addToCart(int id, vector<Product*>& allProducts) {
        items.push_back(p);   // Adds pointer to vector
    }
    bool isEmpty() const { return items.empty(); }
};

// ── Use 3: Item names in Order class ────────────────────────
// Stores string names of items purchased for display in order summary
class Order {
private:
    vector<string> itemNames;
public:
    vector<string> getItemNames() const {
        vector<string> names;
        for (Product* p : items) names.push_back(p->getName());
        return names;
    }
};
```

5.14 Dynamic Memory (new / delete)

Dynamic memory allocation allows objects to be created at run-time on the heap. The new operator allocates memory and calls the constructor; the delete operator calls the destructor and frees the

memory. Dynamic allocation is necessary here because the product objects need to be accessible throughout the entire program — they cannot be stack-allocated inside a limited scope.

Source Code — Dynamic allocation and cleanup in main():

```
// ── Dynamic allocation using 'new' ──────────────────────────
// OOP Concept: Dynamic memory allocation
// These objects are created on the HEAP, not the stack.
// They must be manually deleted to free memory.
vector<Product*> allProducts;
allProducts.push_back(new Electronics(101, "Samsung Galaxy A55", 89999,
"Samsung"));
allProducts.push_back(new Electronics(102, "HP Laptop 15s",     120000, "HP"));
allProducts.push_back(new Electronics(103, "JBL Bluetooth Speaker", 8500,
"JBL"));
allProducts.push_back(new Clothing(201, "Linen Kurta",   2500, "M"));
allProducts.push_back(new Clothing(202, "Denim Jeans",   4200, "L"));
allProducts.push_back(new Clothing(203, "Winter Hoodie", 3800, "XL"));

// Order is also dynamically allocated so it can be created inside a
// switch case and explicitly deleted after the order summary is shown.
Order* order = new Order(1001, "14-04-2025", total, method, cart.getItemNames());
order->displayOrder();
delete order;   // Order destructor fires, then Payment destructor fires

// ── Cleanup block at end of main() ──────────────────────────
// OOP Concept: Dynamic Memory (delete)
// Virtual destructor ensures correct derived destructor runs
cout << "\n[CLEANUP] Deleting all products from memory...\n";
for (Product* p : allProducts) {
    delete p;   // Calls Electronics::~Electronics() or Clothing::~Clothing()
                // then Product::~Product() — correct chain due to virtual dtor
}
allProducts.clear();
cout << "\n[Program End] All objects destroyed. Goodbye!\n";
```

6. OOP CONCEPTS REFERENCE TABLE

The table below maps every OOP concept to its exact class, approximate line range, and a one-line explanation for quick reference.

#	OOP Concept	Class / Location	Lines	Explanation
1	**Abstract Class**	`class Product`	**~36–62**	Pure virtual displayProduct() = 0 makes Product abstract. Cannot be instantiated directly.
2	**Inheritance (Product)**	`Electronics, Clothing`	**~69–112**	Both derive from Product via public inheritance, inheriting productID, name, price.
3	**Runtime Polymorphism**	`displayProduct() override`	**~80–110**	Product* base pointer calls correct Electronics or Clothing version at run-time.
4	**Inheritance (User)**	`Customer, Admin`	**~285–358**	Customer and Admin derive from User, inheriting username and email.
5	**Compile-Time Polymorphism**	`addToCart() overloaded`	**~232–256**	addToCart(int id) and addToCart(string name) resolved at compile time.
6	**Encapsulation**	`All classes`	**Throughout**	Private/protected data with public getters/setters across every class.
7	**Composition**	`Order ← Payment`	**~173–209**	Payment is a direct member of Order — created and destroyed with it.
8	**Aggregation**	`Cart ← Product*`	**~216–278**	Cart holds vector<Product*> but products exist independently.
9	**Friend Function**	`applyDiscount()`	**~156–165**	Directly modifies private totalAmount of Payment without a setter.
10	**Structure (struct)**	`struct OrderDetails`	**~23–26**	Holds orderID and date as a lightweight plain data container.
11	**Constructors**	`All classes`	**Throughout**	Parameterized constructors initialise members and print creation messages.
12	**Destructors**	`All classes`	**Throughout**	Virtual destructors ensure correct chain fires when deleting via base pointer.
13	**STL vector**	`Cart, Order, main()`	**~222, ~190, ~371**	vector<Product*> in Cart; vector<string> in Order; allProducts in main().
14	**Dynamic Memory**	`main() allProducts`	**~376–384**	new used to create products on heap; delete used in cleanup block.

NOTE: Line numbers prefixed with ~ are approximate. They are based on the final OnlineShoppingSystem.cpp submitted alongside this report. Minor variation may occur if whitespace or comments were adjusted.

7. SAMPLE CONSOLE OUTPUT

The following excerpts show the actual output produced when the program is compiled with g++ -std=c++17 and run with the sample product data. These excerpts demonstrate each OOP concept visually through its run-time output.

```
==========================================
   ONLINE SHOPPING SYSTEM
==========================================

[STEP 1] Creating Users...
Enter customer name         : talha
Enter customer email        : talha@NED.edu.pk
Enter customer city/country : karachi
[Constructor] User created: talha
[Constructor] Customer profile created: talha
[Constructor] User created: admin_user
[Constructor] Admin created: admin_user

[STEP 2] Creating Products...
[Constructor] Product created: Samsung Galaxy A55
[Constructor] Electronics created: Samsung Galaxy A55
[Constructor] Product created: HP Laptop 15s
[Constructor] Electronics created: HP Laptop 15s
[Constructor] Product created: JBL Bluetooth Speaker
[Constructor] Electronics created: JBL Bluetooth Speaker
[Constructor] Product created: Linen Kurta
[Constructor] Clothing created: Linen Kurta
[Constructor] Product created: Denim Jeans
[Constructor] Clothing created: Denim Jeans
[Constructor] Product created: Winter Hoodie
[Constructor] Clothing created: Winter Hoodie
[Constructor] Cart created.

==========================================
   MAIN MENU
==========================================
  Logged in as: talha

  1. View Products
  2. Add to Cart (by ID)
  3. Add to Cart (by Name)
  4. View Cart
  5. Place Order
  6. Show User Profiles
```

```
==========================================
   MAIN MENU
==========================================
  Logged in as: talha

  1. View Products
  2. Add to Cart (by ID)
  3. Add to Cart (by Name)
  4. View Cart
  5. Place Order
  6. Show User Profiles
  7. Exit

  Enter your choice: 1

==========================================
   AVAILABLE PRODUCTS
==========================================
  Using Runtime Polymorphism (base class pointer):

  [Electronics] ID: 101 | Name: Samsung Galaxy A55 | Brand: Samsung | Price: Rs.89999
  [Electronics] ID: 102 | Name: HP Laptop 15s | Brand: HP | Price: Rs.120000
  [Electronics] ID: 103 | Name: JBL Bluetooth Speaker | Brand: JBL | Price: Rs.8500
  [Clothing]     ID: 201 | Name: Linen Kurta | Size: M | Price: Rs.2500
  [Clothing]     ID: 202 | Name: Denim Jeans | Size: L | Price: Rs.4200
  [Clothing]     ID: 203 | Name: Winter Hoodie | Size: XL | Price: Rs.3800

==========================================
   MAIN MENU
==========================================
  Logged in as: talha

  1. View Products
  2. Add to Cart (by ID)
  3. Add to Cart (by Name)
  4. View Cart
  5. Place Order
  6. Show User Profiles
  7. Exit
```

```
 Enter your choice: 2

==========================================
  ADD TO CART (by ID)
==========================================
 Enter Product ID: 102
  Added to cart by ID: HP Laptop 15s

==========================================
  MAIN MENU
==========================================
 Logged in as: talha

 1. View Products
 2. Add to Cart (by ID)
 3. Add to Cart (by Name)
 4. View Cart
 5. Place Order
 6. Show User Profiles
 7. Exit

 Enter your choice: 3

==========================================
  ADD TO CART (by Name)
==========================================
 Enter Product Name: Linen Kurta
  Added to cart by Name: Linen Kurta

==========================================
  MAIN MENU
==========================================
 Logged in as: talha

 1. View Products
 2. Add to Cart (by ID)
 3. Add to Cart (by Name)
 4. View Cart
 5. Place Order
 6. Show User Profiles
 7. Exit

 Enter your choice:
```

```
==========================================
   MAIN MENU
==========================================
  Logged in as: talha

  1. View Products
  2. Add to Cart (by ID)
  3. Add to Cart (by Name)
  4. View Cart
  5. Place Order
  6. Show User Profiles
  7. Exit

  Enter your choice: 4

==========================================
   VIEW CART
==========================================

---------- YOUR CART ----------
  [Electronics] ID: 102 | Name: HP Laptop 15s | Brand: HP | Price: Rs.120000
  [Clothing]     ID: 201 | Name: Linen Kurta | Size: M | Price: Rs.2500
  --------------------------------
  Cart Total: Rs.122500
--------------------------------
```

```
  Enter your choice: 5

==========================================
   PLACE ORDER
==========================================
  Payment Methods:
  1. Cash on Delivery
  2. Card Payment
  Choose payment method: 1
[Constructor] Payment object created.
[Constructor] Order #1001 created.

  Apply Discount? (1=Yes / 0=No): 1
  Enter discount %: 20
  Discount of 20% applied! New Total: Rs.98000

========== ORDER SUMMARY ==========
  Order ID  : 1001
  Date      : 14-04-2025
  Items     :
    -> HP Laptop 15s
    -> Linen Kurta
  Payment Method : Cash on Delivery
  Total Amount   : Rs.98000
====================================

   Order placed successfully! Thank you, talha!
[Destructor] Order #1001 destroyed.
[Destructor] Payment object destroyed.

==========================================
   MAIN MENU
==========================================
  Logged in as: talha

  1. View Products
  2. Add to Cart (by ID)
  3. Add to Cart (by Name)
  4. View Cart
  5. Place Order
  6. Show User Profiles
  7. Exit

  Enter your choice:
```

Run Testcases ⊗ 0 ⚠ 0

```
==========================================
   MAIN MENU
==========================================
  Logged in as: talha

  1. View Products
  2. Add to Cart (by ID)
  3. Add to Cart (by Name)
  4. View Cart
  5. Place Order
  6. Show User Profiles
  7. Exit

  Enter your choice: 6

==========================================
   USER PROFILES
==========================================

--- Customer Profile ---
  Username : talha
  Email    : talha@NED.edu.pk
  Address  : karachi

--- Admin Profile ---
  Username : admin_user
  Email    : admin@shop.pk
  Admin Code: ADM-2025
```

```
==========================================
   MAIN MENU
==========================================
  Logged in as: talha

  1. View Products
  2. Add to Cart (by ID)
  3. Add to Cart (by Name)
  4. View Cart
  5. Place Order
  6. Show User Profiles
  7. Exit

  Enter your choice: 7

  Thank you for using Online Shopping System!
  Exiting... (Watch destructors below)


[CLEANUP] Deleting all products from memory...
[Destructor] Electronics destroyed: Samsung Galaxy A55
[Destructor] Product destroyed: Samsung Galaxy A55
[Destructor] Electronics destroyed: HP Laptop 15s
[Destructor] Product destroyed: HP Laptop 15s
[Destructor] Electronics destroyed: JBL Bluetooth Speaker
[Destructor] Product destroyed: JBL Bluetooth Speaker
[Destructor] Clothing destroyed: Linen Kurta
[Destructor] Product destroyed: Linen Kurta
[Destructor] Clothing destroyed: Denim Jeans
[Destructor] Product destroyed: Denim Jeans
[Destructor] Clothing destroyed: Winter Hoodie
[Destructor] Product destroyed: Winter Hoodie

[Program End] All objects destroyed. Goodbye!
[Destructor] Cart destroyed.
[Destructor] Admin destroyed: admin_user
[Destructor] User destroyed: admin_user
[Destructor] Customer destroyed: talha
[Destructor] User destroyed: talha
```

8. FUTURE ENHANCEMENTS

The current system is intentionally simple for educational purposes. The following enhancements could be added in future semesters as OOP knowledge grows:

8.1 Additional Product Categories

New product types such as Books, Furniture, or Groceries can be added by simply creating new classes derived from Product and overriding displayProduct(). No changes to existing code are required — this demonstrates the Open/Closed Principle, which states that a system should be open for extension but closed for modification.

8.2 File Persistence

Products and orders could be saved to text files using C++ file streams (fstream, ofstream, ifstream). This would allow the product catalogue to be loaded from a file at startup and orders to be saved to a log file, providing basic data persistence across program runs.

8.3 User Authentication

The User class could be extended with a private password field and a login() method that compares an entered password against a stored (hashed) value. The Admin class could additionally restrict access to administrative operations such as adding or removing products from the catalogue.

8.4 Exception Handling

try-catch blocks could be added around memory allocation and input parsing operations. For example, if new fails to allocate memory for a product, a std::bad_alloc exception would be thrown and caught, allowing the program to display a user-friendly error message instead of crashing.

8.5 Multiple Orders and Order History

Instead of creating a single Order* and immediately deleting it, a vector<Order*> could be maintained to store order history. This would allow the user to view all past orders, demonstrating further use of STL containers and dynamic memory management.

8.6 Discount Codes and Operator Overloading

The operator+ could be overloaded in the Cart class to allow combining two carts. The operator<< could be overloaded for the Product class to simplify printing. This would introduce two additional OOP concepts: Operator Overloading, which is covered in later semesters.

9. SOURCE CODE :

```
// ============================================================
//   ONLINE SHOPPING SYSTEM USING OBJECT-ORIENTED PROGRAMMING
//   Language  : C++
//   Author    : SE-25096
//   Concepts  : Inheritance, Polymorphism, Abstract Class,
//               Composition, Aggregation, Friend Function,
//               Struct, Constructors, Destructors, STL, Encap.
// ============================================================

#include <iostream>
#include <vector>
#include <string>
using namespace std;

// ============================================================
// SECTION 1: STRUCT
// OOP Concept: Structure to store basic order information
// ============================================================
struct OrderDetails {
   int    orderID;  // Unique ID for each order
   string date;     // Date of order placement
};

// ============================================================
// SECTION 2: ABSTRACT BASE CLASS — Product
// OOP Concept: Abstract Class + Runtime Polymorphism
//              Pure virtual function forces derived classes
//              to provide their own displayProduct()
// ============================================================
class Product {
protected:
   int    productID;   // Encapsulation: protected data
   string name;
   double price;

public:
   // Parameterized Constructor
   Product(int id, string n, double p)
      : productID(id), name(n), price(p) {
      cout << "[Constructor] Product created: " << name << "\n";
   }

   // Pure Virtual Function — makes Product an Abstract Class
   // OOP Concept: Polymorphism (Runtime)
   virtual void displayProduct() = 0;

   // Getters — OOP Concept: Encapsulation
```

```
    int    getID()    const { return productID; }
    string getName()  const { return name; }
    double getPrice() const { return price; }

    // Virtual Destructor — important when using base class pointers
    virtual ~Product() {
        cout << "[Destructor] Product destroyed: " << name << "\n";
    }
};

// ============================================================
// SECTION 3: DERIVED CLASS — Electronics
// OOP Concept: Inheritance (from Product)
//              Polymorphism (displayProduct overridden)
// ============================================================
class Electronics : public Product {
private:
    string brand;

public:
    Electronics(int id, string n, double p, string b)
        : Product(id, n, p), brand(b) {
        cout << "[Constructor] Electronics created: " << name << "\n";
    }

    // OOP Concept: Polymorphism — overriding pure virtual function
    void displayProduct() override {
        cout << "  [Electronics] ID: " << productID
             << " | Name: "  << name
             << " | Brand: " << brand
             << " | Price: Rs." << price << "\n";
    }

    ~Electronics() {
        cout << "[Destructor] Electronics destroyed: " << name << "\n";
    }
};

// ============================================================
// SECTION 4: DERIVED CLASS — Clothing
// OOP Concept: Inheritance (from Product)
//              Polymorphism (displayProduct overridden)
// ============================================================
class Clothing : public Product {
private:
    string size;

public:
    Clothing(int id, string n, double p, string s)
```

```
        : Product(id, n, p), size(s) {
        cout << "[Constructor] Clothing created: " << name << "\n";
    }

    // OOP Concept: Polymorphism — overriding pure virtual function
    void displayProduct() override {
        cout << "  [Clothing]     ID: " << productID
             << " | Name: " << name
             << " | Size: " << size
             << " | Price: Rs." << price << "\n";
    }

    ~Clothing() {
        cout << "[Destructor] Clothing destroyed: " << name << "\n";
    }
};

// ============================================================
// SECTION 5: CLASS — Payment
// OOP Concept: Composition (used inside Order)
//              Friend Function (applyDiscount accesses private)
// ============================================================
class Payment {
private:
    double totalAmount;
    string method;

public:
    // OOP Concept: Friend Function Declaration
    friend void applyDiscount(Payment& pay, double discountPercent);

    Payment(double amount, string m)
        : totalAmount(amount), method(m) {
        cout << "[Constructor] Payment object created.\n";
    }

    double getAmount() const { return totalAmount; }
    string getMethod() const { return method; }

    void displayPayment() const {
        cout << "  Payment Method : " << method << "\n";
        cout << "  Total Amount   : Rs." << totalAmount << "\n";
    }

    ~Payment() {
        cout << "[Destructor] Payment object destroyed.\n";
    }
};
```

```
// ============================================================
// SECTION 6: FRIEND FUNCTION — applyDiscount
// OOP Concept: Friend Function
//              Accesses private member 'totalAmount' of Payment
// ============================================================
void applyDiscount(Payment& pay, double discountPercent) {
    double discountAmount = (pay.totalAmount * discountPercent) / 100;
    pay.totalAmount -= discountAmount;
    cout << "  Discount of " << discountPercent
         << "% applied! New Total: Rs." << pay.totalAmount << "\n";
}

// ============================================================
// SECTION 7: CLASS — Order
// OOP Concept: Composition
//              Order HAS-A Payment object (lives inside Order)
// ============================================================
class Order {
private:
    OrderDetails   details;    // OOP Concept: Struct usage
    Payment        payment;    // OOP Concept: Composition
    vector<string> itemNames;

public:
    Order(int id, string date, double amount,
          string payMethod, vector<string> items)
        : payment(amount, payMethod), itemNames(items) {
        details.orderID = id;
        details.date    = date;
        cout << "[Constructor] Order #" << details.orderID << " created.\n";
    }

    void displayOrder() const {
        cout << "\n========== ORDER SUMMARY ==========\n";
        cout << "  Order ID  : " << details.orderID << "\n";
        cout << "  Date      : " << details.date    << "\n";
        cout << "  Items     :\n";
        for (const string& item : itemNames)
            cout << "    -> " << item << "\n";
        payment.displayPayment();
        cout << "====================================\n";
    }

    Payment& getPayment() { return payment; }

    ~Order() {
        cout << "[Destructor] Order #" << details.orderID << " destroyed.\n";
    }
};
```

```
// ============================================================
// SECTION 8: CLASS — Cart
// OOP Concept: Aggregation
//              Cart stores Product* pointers (does not own them)
// ============================================================
class Cart {
private:
   vector<Product*> items;   // OOP Concept: Aggregation + STL vector

public:
   Cart() { cout << "[Constructor] Cart created.\n"; }

   // OOP Concept: Function Overloading — Version 1 (by ID)
   void addToCart(int id, vector<Product*>& allProducts) {
      for (Product* p : allProducts) {
         if (p->getID() == id) {
            items.push_back(p);
            cout << "   Added to cart by ID: " << p->getName() << "\n";
            return;
         }
      }
      cout << "   Product with ID " << id << " not found.\n";
   }

   // OOP Concept: Function Overloading — Version 2 (by Name)
   void addToCart(string name, vector<Product*>& allProducts) {
      for (Product* p : allProducts) {
         if (p->getName() == name) {
            items.push_back(p);
            cout << "   Added to cart by Name: " << p->getName() << "\n";
            return;
         }
      }
      cout << "   Product \"" << name << "\" not found.\n";
   }

   void viewCart() const {
      if (items.empty()) { cout << "  Your cart is empty!\n"; return; }
      cout << "\n---------- YOUR CART ----------\n";
      double total = 0;
      for (Product* p : items) {
         p->displayProduct();   // OOP Concept: Runtime Polymorphism
         total += p->getPrice();
      }
      cout << "  --------------------------------\n";
      cout << "  Cart Total: Rs." << total << "\n";
      cout << "--------------------------------\n";
   }
```

```
    double getTotal() const {
        double total = 0;
        for (Product* p : items) total += p->getPrice();
        return total;
    }

    vector<string> getItemNames() const {
        vector<string> names;
        for (Product* p : items) names.push_back(p->getName());
        return names;
    }

    bool isEmpty() const { return items.empty(); }

    ~Cart() { cout << "[Destructor] Cart destroyed.\n"; }
};

// ============================================================
// SECTION 9: BASE CLASS — User
// OOP Concept: Inheritance (base class) + Encapsulation
// ============================================================
class User {
protected:
    string username;
    string email;

public:
    User(string u, string e) : username(u), email(e) {
        cout << "[Constructor] User created: " << username << "\n";
    }

    virtual void showProfile() const {
        cout << "  Username : " << username << "\n";
        cout << "  Email    : " << email    << "\n";
    }

    string getUsername() const { return username; }

    virtual ~User() {
        cout << "[Destructor] User destroyed: " << username << "\n";
    }
};

// ============================================================
// SECTION 10: DERIVED CLASS — Customer
// OOP Concept: Inheritance (from User)
// ============================================================
class Customer : public User {
```

```
private:
    string address;

public:
    Customer(string u, string e, string addr)
        : User(u, e), address(addr) {
        cout << "[Constructor] Customer profile created: " << username << "\n";
    }

    void showProfile() const override {
        cout << "\n--- Customer Profile ---\n";
        User::showProfile();
        cout << "  Address  : " << address << "\n";
    }

    ~Customer() {
        cout << "[Destructor] Customer destroyed: " << username << "\n";
    }
};

// ============================================================
// SECTION 11: DERIVED CLASS — Admin
// OOP Concept: Inheritance (from User)
// ============================================================
class Admin : public User {
private:
    string adminCode;

public:
    Admin(string u, string e, string code)
        : User(u, e), adminCode(code) {
        cout << "[Constructor] Admin created: " << username << "\n";
    }

    void showProfile() const override {
        cout << "\n--- Admin Profile ---\n";
        User::showProfile();
        cout << "  Admin Code: " << adminCode << "\n";
    }

    ~Admin() {
        cout << "[Destructor] Admin destroyed: " << username << "\n";
    }
};

// ============================================================
// HELPER FUNCTIONS
// These keep main() clean by moving each menu action here
// ============================================================
```

```
// Prints a styled separator header
void printHeader(const string& title) {
    cout << "\n==========================================\n";
    cout << "   " << title << "\n";
    cout << "==========================================\n";
}

// Loads all products into the catalogue using dynamic memory
// OOP Concept: Dynamic Memory (new), Abstract Class, Polymorphism
void loadProducts(vector<Product*>& allProducts) {
    cout << "\n[STEP 2] Creating Products...\n";
    allProducts.push_back(new Electronics(101, "Samsung Galaxy A55",    89999,  "Samsung"));
    allProducts.push_back(new Electronics(102, "HP Laptop 15s",         120000, "HP"));
    allProducts.push_back(new Electronics(103, "JBL Bluetooth Speaker",  8500,  "JBL"));
    allProducts.push_back(new Clothing   (201, "Linen Kurta",           2500,  "M"));
    allProducts.push_back(new Clothing   (202, "Denim Jeans",           4200,  "L"));
    allProducts.push_back(new Clothing   (203, "Winter Hoodie",         3800,  "XL"));
}

// Frees all heap-allocated products
// OOP Concept: Dynamic Memory (delete) + Virtual Destructor chain
void cleanupProducts(vector<Product*>& allProducts) {
    cout << "\n[CLEANUP] Deleting all products from memory...\n";
    for (Product* p : allProducts)
        delete p;
    allProducts.clear();
    cout << "\n[Program End] All objects destroyed. Goodbye!\n";
}

// Displays all products using runtime polymorphism
// OOP Concept: Runtime Polymorphism (Product* calls correct override)
void handleViewProducts(vector<Product*>& allProducts) {
    printHeader("AVAILABLE PRODUCTS");
    cout << "  Using Runtime Polymorphism (base class pointer):\n\n";
    for (Product* p : allProducts)
        p->displayProduct();
}

// Adds a product to cart by ID (overloaded version 1)
// OOP Concept: Function Overloading (Compile-Time Polymorphism)
void handleAddByID(Cart& cart, vector<Product*>& allProducts) {
    printHeader("ADD TO CART (by ID)");
    cout << "  Enter Product ID: ";
    int id;
    cin >> id;
    cart.addToCart(id, allProducts);
}
```

```
// Adds a product to cart by Name (overloaded version 2)
// OOP Concept: Function Overloading (Compile-Time Polymorphism)
void handleAddByName(Cart& cart, vector<Product*>& allProducts) {
    printHeader("ADD TO CART (by Name)");
    cin.ignore();
    cout << "  Enter Product Name: ";
    string name;
    getline(cin, name);
    cart.addToCart(name, allProducts);
}

// Places an order with Composition, Struct, and Friend Function
// OOP Concept: Composition (Order HAS-A Payment)
//              Struct (OrderDetails inside Order)
//              Friend Function (applyDiscount)
//              Dynamic Memory (new Order, delete order)
void handlePlaceOrder(Cart& cart, const string& customerName) {
    if (cart.isEmpty()) {
        cout << "\n   Cart is empty! Add products first.\n";
        return;
    }

    printHeader("PLACE ORDER");

    cout << "  Payment Methods:\n";
    cout << "  1. Cash on Delivery\n";
    cout << "  2. Card Payment\n";
    cout << "  Choose payment method: ";
    int pm;
    cin >> pm;
    string method = (pm == 2) ? "Card Payment" : "Cash on Delivery";

    // OOP Concept: Composition — Order is created with Payment inside it
    Order* order = new Order(1001, "14-04-2025",
                             cart.getTotal(), method,
                             cart.getItemNames());

    // OOP Concept: Friend Function — applyDiscount accesses private Payment data
    cout << "\n  Apply Discount? (1=Yes / 0=No): ";
    int applyD;
    cin >> applyD;
    if (applyD == 1) {
        cout << "  Enter discount %: ";
        double disc;
        cin >> disc;
        applyDiscount(order->getPayment(), disc);
    }

    order->displayOrder();
```

```
    cout << "\n   Order placed successfully! Thank you, " << customerName << "!\n";

    delete order;   // OOP Concept: Dynamic Memory (delete)
}

// Shows profiles of both Customer and Admin
// OOP Concept: Inheritance + Runtime Polymorphism (showProfile override)
void handleShowProfiles(const Customer& customer, const Admin& admin) {
    printHeader("USER PROFILES");
    customer.showProfile();
    admin.showProfile();
}

// Prints the main menu options
void printMenu(const string& username) {
    printHeader("MAIN MENU");
    cout << "  Logged in as: " << username << "\n\n";
    cout << "  1. View Products\n";
    cout << "  2. Add to Cart (by ID)\n";
    cout << "  3. Add to Cart (by Name)\n";
    cout << "  4. View Cart\n";
    cout << "  5. Place Order\n";
    cout << "  6. Show User Profiles\n";
    cout << "  7. Exit\n";
    cout << "\n  Enter your choice: ";
}

// ============================================================
// MAIN FUNCTION — Clean and simple
// All logic is delegated to helper functions above
// ============================================================
int main() {

    printHeader("ONLINE SHOPPING SYSTEM");

    // ── Create Customer (OOP: Inheritance) ──────────────────
    cout << "\n[STEP 1] Creating Users...\n";
    string name, email, city;
    cout << "Enter customer name         : "; cin >> name;
    cout << "Enter customer email        : "; cin >> email;
    cout << "Enter customer city/country : "; cin >> city;

    Customer customer(name, email, city);
    Admin    admin("admin_user", "admin@shop.pk", "ADM-2025");

    // ── Load Products (OOP: Dynamic Memory, Polymorphism) ────
    vector<Product*> allProducts;
    loadProducts(allProducts);
```

```
    // ── Create Cart (OOP: Aggregation) ──────────────────────────
    Cart cart;

    // ── Menu Loop ──────────────────────────────────────────────────────────
    int  choice  = 0;
    bool running = true;

    while (running) {
        printMenu(customer.getUsername());
        cin >> choice;

        switch (choice) {
            case 1: handleViewProducts(allProducts);                    break;
            case 2: handleAddByID(cart, allProducts);                   break;
            case 3: handleAddByName(cart, allProducts);                 break;
            case 4: printHeader("VIEW CART"); cart.viewCart();          break;
            case 5: handlePlaceOrder(cart, customer.getUsername());     break;
            case 6: handleShowProfiles(customer, admin);                break;
            case 7: running = false;
                    cout << "\n  Thank you for using Online Shopping System!\n";
                    cout << "  Exiting... (Watch destructors below)\n\n";    break;
            default: cout << "\n   Invalid choice! Please try again.\n";
        }
    }

    // ── Cleanup (OOP: Dynamic Memory delete) ────────────────────
    cleanupProducts(allProducts);

    return 0;
}

// ============================================================
// OOP CONCEPTS SUMMARY (Quick Reference)
// ------------------------------------------------------------
// 1.  Inheritance       : User -> Customer, Admin
//                         Product -> Electronics, Clothing
// 2.  Polymorphism      : displayProduct() virtual, overridden
//                         showProfile() virtual, overridden
// 3.  Abstract Class    : Product (pure virtual displayProduct)
// 4.  Encapsulation     : private variables + getters/setters
// 5.  Composition       : Order HAS-A Payment (inside Order)
// 6.  Aggregation       : Cart stores Product* (external objs)
// 7.  Friend Function   : applyDiscount accesses Payment private
// 8.  Struct            : OrderDetails { orderID, date }
// 9.  Constructors      : All classes have constructors
// 10. Destructors       : All classes print on destruction
// 11. Function Overload : addToCart(int) & addToCart(string)
// 12. STL vector        : vector<Product*>, vector<string>
// 13. Dynamic Memory    : new Product, delete p
```

9. CONCLUSION

This project successfully demonstrates all fourteen required Object-Oriented Programming concepts within a single, well-structured, and beginner-friendly C++ console application. The Online Shopping System covers Abstract Class, Inheritance (two separate hierarchies), Runtime Polymorphism, Compile-Time Polymorphism through Function Overloading, Encapsulation, Composition, Aggregation, Friend Function, Structure (struct), Constructors, Destructors, STL vector, and Dynamic Memory Management.

The two inheritance hierarchies — Product leading to Electronics and Clothing, and User leading to Customer and Admin — demonstrate how code reuse and extensibility are achieved through inheritance. The abstract Product class enforces correct usage by preventing generic products from being created directly. The vector<Product*> demonstrates runtime polymorphism in the most practical way: a loop over mixed object types calling the correct function automatically.

The Composition relationship between Order and Payment shows how object ownership is modelled in OOP, while the Aggregation relationship between Cart and Products shows how objects can be shared and referenced without ownership. The friend function applyDiscount() provides a controlled exception to encapsulation — granting one trusted function direct access to private data.

The visible constructor and destructor messages on the console give first-year students a concrete, real-time view of the C++ object lifecycle — from creation through destruction. The virtual destructor chain on deletion through base-class pointers, the composition-based automatic destruction of Payment when Order is deleted, and the stack-based automatic destruction of all local objects at program exit together demonstrate all the key ways C++ manages object lifetime.

This project provides a strong and practical foundation for understanding OOP in C++. The design decisions made here — abstract base classes, the correct choice between Composition and Aggregation, encapsulation of sensitive data, and the use of virtual destructors — are patterns that software engineering students will encounter in industry-level codebases throughout their careers.